**Instituto Politécnico do Cávado e do Ave**
**Escola Superior de Tecnologia**

# Trabalho Prático I

Deteção de Fraude em Transações Financeiras

*Unidade Curricular: Integração de Sistemas de Informação*

**Bruno Paiva**
a31496@alunos.ipca.pt

Licenciatura em Engenharia de Sistemas Informáticos

Outubro de 2026

**Resumo**

Este relatório descreve o desenvolvimento de um processo ETL para deteção de transações financeiras suspeitas. A solução integra dados de diferentes formatos e origens, transforma e enriquece os registos com informação de cartões, comerciantes, países e taxas de câmbio, e encaminha os resultados para uma base de dados, ficheiros de exportação, notificações por email e dashboards de monitorização.

A implementação combina transações históricas em ficheiro CSV com telemetria contínua em streaming simulada através de fluxos Node-RED e transporte por MQTT (Eclipse Mosquitto). A lógica analítica foi concebida no KNIME Analytics Platform, incluindo normalização por expressões regulares, mascaramento de PAN, validação do algoritmo de Luhn, conversão de moeda, cálculo de $Z$-score comportamental e um motor de regras que gera uma pontuação de risco. Os resultados são persistidos em PostgreSQL, com alertas exportados para CSV/JSON, notificações automáticas enviadas via MailHog e visualização analítica em Grafana. Todo o processo é gerido através de um workflow orquestrador.

# Índice


**1 Enquadramento** **2**
1.1 Objetivos . . . 2

**2 Problema** **4**

**3 Estratégia utilizada** **5**
3.1 Tecnologias utilizadas . . . 5
3.2 Fontes de dados . . . 5
3.3 Arquitetura geral . . . 6

**4 Transformações** **7**
4.1 Fase 1: recolha e preparação de dados (`01_ingestao`) . . . 7
4.2 Fase 2: processamento e transformação (`02_transformacao`) . . . 8
4.2.1 Limpeza e tratamento de inconsistências . . . 8
4.2.2 Normalização de comerciantes com Expressões Regulares (Regex) . . . 8
4.2.3 Validação do algoritmo de Luhn (ISO/IEC 7812) . . . 8
4.2.4 Mascaramento do PAN . . . 9
4.2.5 Cruzamentos e enriquecimento heterogéneo . . . 9
4.2.6 Cálculo de indicadores comportamentais e $Z$-score . . . 10
4.2.7 Motor de regras e cálculo do Score de Risco . . . 10
4.3 Fase 3: persistência, exportação e notificação (`03_carga_notificacao`) . . . 11
4.3.1 Resultados da execução da pipeline . . . 11
4.4 Dashboard de monitorização . . . 12
4.4.1 Alertas por hora . . . 12
4.4.2 Top 5 países de risco . . . 12
4.4.3 Valor total e valor sinalizado por dia . . . 13

**5 Jobs** **14**

**6 Vídeo de demonstração** **16**

**7 Conclusão e trabalhos futuros** **17**
7.1 Decisões de âmbito e exclusões assumidas . . . 17
7.2 Trabalhos futuros . . . 18

**8 Referências bibliográficas** **19**

# Capítulo 1

# Enquadramento

O presente trabalho foi desenvolvido no âmbito da unidade curricular de Integração de Sistemas de Informação, lecionada no 1.º semestre do 3.º ano da Licenciatura em Engenharia de Sistemas Informáticos da Escola Superior de Tecnologia do Instituto Politécnico do Cávado e do Ave.

O trabalho foi realizado em regime individual por Bruno Paiva, estudante n.º 31496. O tema escolhido consiste na deteção de fraude em transações financeiras através de um processo de ETL (*Extract, Transform, Load*).

O enunciado do Trabalho Prático I propõe a aplicação prática de conceitos de integração de sistemas ao nível dos dados, explorando a recolha a partir de origens heterogéneas, o processamento e transformação da informação com recurso a expressões regulares, regras de negócio e cálculos estatísticos, bem como a persistência em bases de dados relacionais, exportação de ficheiros e mecanismos de alerta e visualização [1].

## 1.1 Objetivos

Os principais objetivos delineados para este projeto são:

- recolher e integrar dados de diferentes formatos (CSV, JSON, XML) e origens heterogéneas (ficheiros locais, base de dados relacional e streaming via broker MQTT);
- efetuar a limpeza e normalização dos registos, eliminando duplicados e tratando inconsistências de formato;
- aplicar técnicas de higienização de texto através de expressões regulares (Regex);
- implementar salvaguardas de proteção de dados sensíveis, incluindo o mascaramento do número do cartão (PAN);
- validar a integridade dos números de cartão de pagamento através do algoritmo de Luhn;
- enriquecer as transações com perfis de cartões, categorias e níveis de risco de comerciantes, taxas diárias de câmbio e risco de países;
- calcular indicadores comportamentais, nomeadamente médias, desvios e o desvio padronizado ($Z$-score);

- definir e aplicar um motor de regras ponderado para atribuir um índice de suspeita a cada operação e filtrar alertas críticos;
- persistir os dados resultantes num esquema relacional em PostgreSQL estruturado em tabelas de staging e data mart;
- exportar os alertas para ficheiros CSV e JSON e expedir notificações por email através do MailHog;
- disponibilizar visualizações analíticas em tempo real num dashboard Grafana;
- orquestrar sequencialmente as diferentes fases do processo ETL através de um workflow mestre de controlo.

# Capítulo 2

# Problema

As instituições bancárias e os operadores de meios de pagamento processam diariamente milhões de transações eletrónicas com origem em múltiplos canais (terminais físicos POS, comércio eletrónico e caixas automáticas). A rápida identificação de operações fraudulentas é fundamental para conter perdas financeiras sem interromper indevidamente as transações legítimas dos clientes.

Na prática operacional, a informação chega de forma dispersa: lotes históricos consolidados no final do dia e fluxos contínuos de transações emitidas a cada segundo pelos terminais de pagamento. Além disso, os dados brutos apresentam frequentemente problemas de qualidade, tais como identificadores repetidos, nomes de comerciantes despadronizados, valores em moedas diferentes e numerações de cartões potencialmente incorretas.

O problema abordado consiste em construir um pipeline ETL integrado que receba estas origens, proceda à sua homogeneização e deduplicação, cruze cada operação com informação cadastral e contextual de apoio, e avalie o risco de cada movimento com base em regras de negócio e desvios ao perfil habitual do cliente. Os resultados devem ser armazenados para arquivo histórico e auditoria, com aviso imediato das transações classificadas como de elevado risco.

Para viabilizar o desenvolvimento e a experimentação sem recorrer a informação bancária confidencial, foi utilizado um conjunto de dados sintético. Os dados foram estruturados de forma a incorporar propositadamente casos patológicos para validação do sistema: transações duplicadas, valores em falta, nomes com pontuação e siglas societárias irregulares, transações concentradas em intervalos curtos de tempo, operações noturnas em países de risco e números de cartão que falham o dígito de controlo matemático.

# Capítulo 3

# Estratégia utilizada

A arquitetura da solução foi estruturada de forma modular, dividindo as operações pelas três fases canónicas do ciclo ETL: ingestão (`01_ingestao`), transformação (`02_transformacao`) e carga/notificação (`03_carga_notificacao`). Um quarto workflow, `00_orquestrador`, coordena a execução sequencial do conjunto.

## 3.1 Tecnologias utilizadas

Para suportar o pipeline foram selecionadas as seguintes ferramentas:

- **KNIME Analytics Platform (v5.12):** plataforma principal para construção e execução visual dos workflows de transformação e orquestração;
- **PostgreSQL 16:** sistema de gestão de base de dados relacional para alojar as tabelas de staging, dimensões e os data marts de alertas e resumos diários;
- **Docker Compose:** orquestração de contentores para arranque local, reprodutível e isolado de todos os serviços de suporte;
- **Node-RED:** simulação de terminais POS através da geração periódica de mensagens JSON e encaminhamento das mensagens recebidas para a base de dados;
- **Eclipse Mosquitto (MQTT):** broker leve de mensagens para transporte telemétrico assíncrono das transações em streaming;
- **Grafana:** plataforma de observabilidade conectada ao PostgreSQL para apresentação de métricas analíticas e painéis de monitorização;
- **MailHog:** servidor SMTP local para receção e visualização de emails de alerta sem necessidade de ligações a servidores externos.

## 3.2 Fontes de dados

O Quadro 3.1 detalha as fontes de dados heterogéneas utilizadas pelo processo.

| Fonte | Formato | Utilização |
|---|---|---|
| `transacoes.csv` | CSV (10 447 linhas) | Lote histórico de transações de 30 dias (10/set a 10/out). |
| `stg_stream` | PostgreSQL / JSONB | Tabela alimentada em tempo real com mensagens do tópico MQTT `fraude/transacoes`. |
| `cartoes.json` | JSON (500 cartões) | Cadastro de cartões de débito/crédito com limites e nomes dos clientes. |
| `regras.xml` | XML (10 categorias) | Catálogo hierárquico com código MCC, categoria e indicador de risco. |
| `paises_risco.csv` | CSV (16 países) | Classificação de risco geopolítico por país. |
| `cambio.json` | JSON (90 taxas) | Séries diárias de taxas de conversão para euros (USD, BRL e EUR). |

Tabela 3.1: Fontes de dados de entrada heterogéneas utilizadas no projeto.

Esta composição assegura a diversidade de formatos requerida pelo enunciado (três formatos de ficheiro: CSV, JSON e XML, para além de registos em base de dados e payloads JSONB em fluxo contínuo).

## 3.3 Arquitetura geral

A Figura 3.1 ilustra o fluxo de dados entre os componentes do ecossistema.

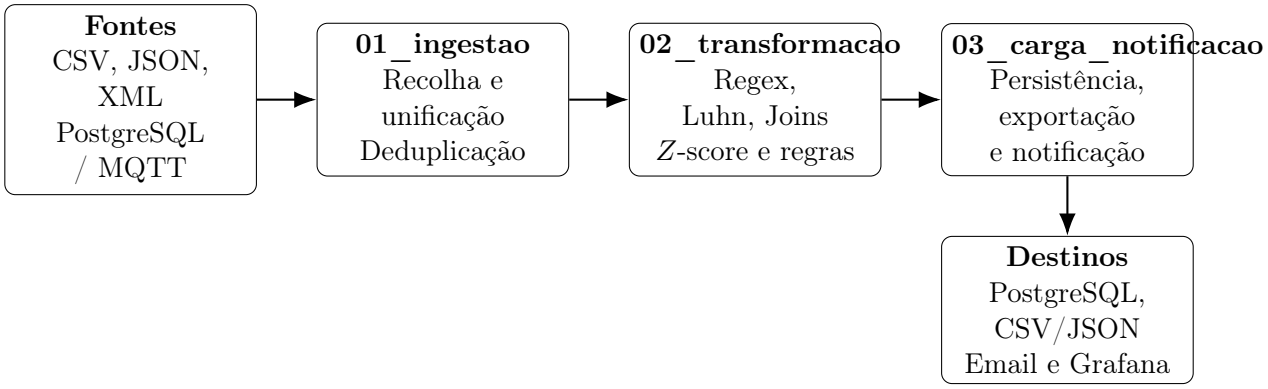



Figura 3.1: Arquitetura geral do processo ETL.

# Capítulo 4

# Transformações

## 4.1 Fase 1: recolha e preparação de dados (`01_ingestao`)

O workflow `01_ingestao` tem como objetivo extrair os dados das fontes de transações (lote e fluxo contínuo), uniformizar as estruturas e consolidar as transações limpas na tabela `stg_ingestao` da base de dados PostgreSQL.

O processo divide-se em dois ramos de leitura:

1. **Ramo em Lote (CSV):** o node `CSV Reader` lê o ficheiro histórico `transacoes.csv`. Seguem-se os nodes `Number to String` e `String to Date&Time` para converter a coluna temporal de texto para data (`yyyy-MM-dd HH:mm:ss`). O node `String Manipulation` atribui a coluna de proveniência `origem = "csv"`.

2. **Ramo em Streaming (PostgreSQL):** o simulador no Node-RED publica transações a cada 2 segundos no tópico MQTT `fraude/transacoes`. O próprio fluxo Node-RED insere o payload JSON recebido na tabela `stg_stream`. No KNIME, os nodes `PostgreSQL Connector`, `DB Table Selector` e `DB Reader` acedem à tabela executando a seguinte consulta de extração dos campos de JSONB:

```sql
SELECT
  ((payload::jsonb #>> '{}')::jsonb ->> 'id') AS id_transacao,
  ((payload::jsonb #>> '{}')::jsonb ->> 'ts') AS data_hora,
  ((payload::jsonb #>> '{}')::jsonb ->> 'cartao') AS id_cartao,
  ((payload::jsonb #>> '{}')::jsonb ->> 'mcc') AS mcc,
  ((payload::jsonb #>> '{}')::jsonb ->> 'valor')::numeric AS valor,
  ((payload::jsonb #>> '{}')::jsonb ->> 'pais') AS pais
FROM stg_stream;
```

Listing 4.1: Consulta para extração dos atributos do payload JSONB da tabela stg_stream.

   O node `String to Date&Time` converte o formato ISO-8601 (`yyyy-MM-dd'T'HH:mm:ss.SSSXXX`) e o node `String Manipulation` marca o campo `origem = "stream"`.

De seguida, o node `Concatenate` unifica ambos os ramos. Antes da escrita na base de dados, é imperativo aplicar o node `Duplicate Row Filter` sobre o identificador `id_transacao`. Uma vez que o identificador constitui a chave primária da tabela destino, a remoção prévia

de duplicados evita violações de unicidade no PostgreSQL. Foram identificados e expurgados 197 registos duplicados, gravando-se no final um total de **10 428 transações únicas** na tabela `stg_ingestao` (10 250 provenientes do ficheiro CSV e 178 do streaming) através do node `DB Writer`.

## 4.2 Fase 2: processamento e transformação (`02_transformacao`)

O workflow `02_transformacao` efetua a leitura da tabela `stg_ingestao` e implementa as transformações de limpeza, segurança, enriquecimento relacional e avaliação analítica de risco.

### 4.2.1 Limpeza e tratamento de inconsistências

Os nodes `DB Table Selector` e `DB Reader` carregam os dados para memória. O node `Number Rounder` arredonda os montantes para duas casas decimais e os nodes `Row Filter` descartam linhas inválidas que não possuam identificador de cartão associado.

### 4.2.2 Normalização de comerciantes com Expressões Regulares (Regex)

Os nomes dos estabelecimentos comerciais chegam com variações de grafia, sufixos societários e pontuação excessiva. No node `String Manipulation`, foi configurada a seguinte expressão regular composta:

```
strip(regexReplace(regexReplace(upperCase($comerciante$), "\\bLDA\\b|S\\.A\\.|\\
    p{Punct}", ""), "\\s+", " "))
```

Listing 4.2: Expressão regular para limpeza e padronização dos comerciantes.

Esta transformação converte todo o texto para maiúsculas, elimina menções a `LDA`, `S.A.` e carateres de pontuação (`\p{Punct}`), e substitui múltiplos espaços em branco sucessivos por um espaço simples.

### 4.2.3 Validação do algoritmo de Luhn (ISO/IEC 7812)

Para detetar erros de digitação e números de cartão forjados, foi integrado um node `Java Snippet` que executa a validação do dígito de controlo do PAN segundo o algoritmo de Luhn:

```
String pan = getCell("pan", tString);
boolean valid = false;

if (pan != null && pan.matches("\\d{13,19}")) {
    int sum = 0;
    boolean doubleDigit = false;

    // Percorre os digitos da direita para a esquerda
    for (int i = pan.length() - 1; i >= 0; i--) {
        int digit = pan.charAt(i) - '0';
```

```
11
12          if (doubleDigit) {
13              digit *= 2;
14              if (digit > 9) {
15                  digit -= 9;
16              }
17          }
18
19          sum += digit;
20          doubleDigit = !doubleDigit;
21      }
22
23      valid = (sum % 10 == 0);
24  }
25
26  out_isValidLuhn = valid;
```

Listing 4.3: Código Java para cálculo e validação do dígito de controlo de Luhn.

O resultado booleano é guardado na nova coluna `out_isValidLuhn`, permitindo sinalizar imediatamente cartões que não cumprem o padrão numérico oficial.

### 4.2.4 Mascaramento do PAN

Por razões de segurança e boas práticas na manipulação de cartões bancários, o número integral do cartão não deve circular nos relatórios operacionais. O node `String Manipulation` extrai apenas os últimos 4 dígitos para o campo `pan_mascara`:

```
1  substr($pan$, length($pan$) - 4, 4)
```

Listing 4.4: Fórmula para retenção exclusiva dos últimos 4 dígitos do cartão.

### 4.2.5 Cruzamentos e enriquecimento heterogéneo

O workflow enriquece a transação cruzando-a com quatro origens distintas:

- **Cartões (JSON):** `JSON Reader` e `JSON Path` lêem `cartoes.json`; o node `Joiner` cruza pela chave `id_cartao` trazendo o nome do cliente, o tipo de cartão e o limite de crédito;
- **Regras MCC (XML):** `XML Reader` e `XPath` lêem `regras.xml`; o node `Joiner` cruza por `mcc` adicionando a categoria comercial e o indicador de risco setorial (`risco = 1` para categorias como Apostas, Criptomoedas, Caixas Multibanco e Transferências);
- **Taxas de Câmbio (JSON):** `JSON Reader` e `JSON Path` lêem `cambio.json`; o node `Date_Time to String` obtém a data da transação (`dia`) e o node `Joiner` associa a taxa diária através da chave composta `(dia, moeda)`. Um node `Java Snippet` calcula o valor convertido em euros:

```
Double valor = getCell("valor", tDouble);
Double taxa = getCell("taxa_eur", tDouble);

if (valor == null || taxa == null || taxa == 0.0) {
    out_valor_eur = null;
} else {
    out_valor_eur = valor / taxa;
}
```

Listing 4.5: Conversão do valor da transação para euros.

- **Países de Risco (CSV):** `CSV Reader` carrega `paises_risco.csv` e o node `Joiner` cruza pela coluna `pais`, identificando transações efetuadas em territórios classificados com risco elevado.

### 4.2.6 Cálculo de indicadores comportamentais e $Z$-score

Para avaliar se o montante transacionado é anómalo para determinado titular:

1. o node `GroupBy` calcula a média amostral ($\mu$) e o desvio padrão ($\sigma$) do campo `valor_eur` agrupando por `id_cartao`;
2. o node `Joiner` junta estas duas métricas de volta a cada registo da transação;
3. o node `Math Formula` calcula o valor do $Z$-score através da fórmula:

$$Z = \frac{\text{valor_eur} - \mu_{\text{cartão}}}{\sigma_{\text{cartão}}} \tag{4.1}$$

   Um valor de $Z > 3$ traduz uma operação que se desvia mais de 3 desvios padrão da média habitual daquele cartão, constituindo um forte indício de anomalia;
4. o node `Date_Time Part Extractor` obtém a hora da transação (`hora` entre 0 e 23) para escrutínio de operações na madrugada.

### 4.2.7 Motor de regras e cálculo do Score de Risco

No node `Expression`, foi implementada a pontuação ponderada de suspeita através da seguinte fórmula aditiva:

```
if($z_score > 3, 30, 0)
+ if($transacoes_10min >= 5, 30, 0)
+ if($["risco (Right)"] = 1 and $hora < 5, 25, 0)
+ if($risco = "1", 15, 0)
+ if(not $out_isValidLuhn, 10, 0)
```

Listing 4.6: Fórmula de cálculo do Score de Risco no node Expression.

A ponderação de cada fator reparte-se da seguinte forma:

- **Valor acima de 3 desvios padrão ($Z > 3$):** 30 pontos (desvio excessivo face ao perfil histórico do titular);

- **Velocidade elevada ($\geq$ 5 transações em 10 minutos):** 30 pontos (possível ataque de força bruta ou utilização abusiva contínua);
- **País de risco na madrugada (entre as 00h e as 05h):** 25 pontos (operações noturnas em territórios suspeitos);
- **Comerciante de categoria sensível (MCC de risco):** 15 pontos (comércio associado a risco de fraude ou lavagem);
- **Número de cartão inválido (falha de Luhn):** 10 pontos (cartão adulterado ou incorreto).

O node `Rule-based Row Splitter` separa os alertas com a condição `$score$ >= 50 => TRUE`. As operações com pontuação igual ou superior a 50 pontos seguem para a porta superior de alertas, sendo ordenadas por ordem decrescente de pontuação através do node `Sorter`.

## 4.3 Fase 3: persistência, exportação e notificação (`03_carga_noti`

O workflow `03_carga_notificacao` recebe os dados transformados e procede ao encaminhamento para os diversos destinos:

- **Tabela `mart_alertas`:** o node `DB Merger` atualiza ou insere transações com `score` $\geq$ 50 baseando-se na chave `id_transacao`, garantindo a idempotência de execuções sucessivas;
- **Tabela `mart_resumo_diario`:** uma combinação de nodes `GroupBy`, `Joiner`, `Missing Value`, `Column Renamer` e `DB Writer` agrega os dados por data e por país, registando o total de transações, número de alertas e volume financeiro em euros;
- **Ficheiros de Exportação:** o node `CSV Writer` guarda os alertas no ficheiro `data/output/alertas.csv` e os nodes `Table to JSON` e `JSON Writer` gravam os mesmos registos em `data/output/alertas0.json`;
- **Notificação por Email:** o node `GroupBy` apura a quantidade de alertas registados. Essa contagem é convertida em variável de fluxo pelo node `Table Row to Variable` e transmitida ao node `Send Email`, que remete uma mensagem automática para o servidor local **MailHog** (`localhost:1025`) com o assunto `"Alertas de fraude"` e corpo de texto com o número de alertas sinalizados.

### 4.3.1 Resultados da execução da pipeline

Após a execução completa da pipeline sobre o volume de dados de teste, o sistema produziu os seguintes resultados concretos:

- **Transações Ingeridas:** 10 428 registos consolidados em `stg_ingestao`;
- **Alertas Detetados:** foram identificadas com precisão **4 transações com pontuação de risco** $\geq$ 50:

  - `TXB010426` (Score 75, valor 401,75 €, hora 13, MCC de risco 4829, 5 transações em 10 min, $Z = 3,31$);
  - `TXB010436` (Score 60, valor 361,50 €, hora 15, 5 transações em 10 min, $Z = 3,60$);
  - `TXB010417` (Score 60, valor 391,93 €, hora 19, 5 transações em 10 min, $Z = 4,21$);
  - `TXB010281` (Score 60, valor 397,25 €, hora 06, 5 transações em 10 min, $Z = 3,37$).

- **Resumo Diário:** 274 registos agregados por país e dia na tabela `mart_resumo_diario`;
- **Notificação:** email intercetado com sucesso no MailHog com o aviso *"Foram identificados 4 alertas."*.

## 4.4 Dashboard de monitorização

A dashboard Grafana consulta as tabelas PostgreSQL para apresentar os resultados do processo. Foram definidas três visualizações: alertas por hora, países de risco com mais alertas e comparação entre o valor total e o valor sinalizado por dia.

### 4.4.1 Alertas por hora

A visualização utiliza o formato *Time series*.

```sql
SELECT date_trunc('hour', s.data_hora) AS "time",
       COUNT(*) AS alertas
FROM mart_alertas a
JOIN stg_ingestao s USING (id_transacao)
GROUP BY 1
ORDER BY 1;
```

Listing 4.7: Consulta para alertas agregados por hora.

### 4.4.2 Top 5 países de risco

A visualização utiliza o formato *Bar chart*.

```sql
SELECT p.pais,
       p.risco,
       SUM(r.n_alertas) AS alertas
FROM mart_resumo_diario r
JOIN ref_pais_risco p USING (pais)
GROUP BY p.pais, p.risco
ORDER BY p.risco DESC, alertas DESC
LIMIT 5;
```

Listing 4.8: Consulta para os cinco países de risco com mais alertas.

### 4.4.3 Valor total e valor sinalizado por dia

A visualização utiliza o formato *Time series*.

```sql
WITH total AS (
  SELECT dia, SUM(valor_eur) AS valor_total_eur
  FROM mart_resumo_diario
  GROUP BY dia
),
sinalizado AS (
  SELECT s.data_hora::date AS dia,
         SUM(a.valor_eur) AS valor_sinalizado_eur
  FROM mart_alertas a
  JOIN stg_ingestao s USING (id_transacao)
  GROUP BY 1
)
SELECT total.dia::timestamp AS "time",
       total.valor_total_eur,
       COALESCE(sinalizado.valor_sinalizado_eur, 0)
           AS valor_sinalizado_eur
FROM total
LEFT JOIN sinalizado USING (dia)
ORDER BY 1;
```

Listing 4.9: Consulta para comparação dos valores diários.

# Capítulo 5

# Jobs

O conceito tradicional de Job nas ferramentas de ETL (como o Pentaho Kettle ou o Talend) corresponde ao elemento de controlo de processos (*Process Control*) responsável por gerir a sequência de execução de várias transformações dependentes, assegurando validações de fluxo, tratamento de exceções e integridade global.

No KNIME Analytics Platform, essa função é desempenhada de forma equivalente através de um workflow orquestrador mestre. O workflow `00_orquestrador` representa o job principal do projeto, coordenando a chamada sequencial dos workflows subordinados através de nodes `Call Workflow Service`.

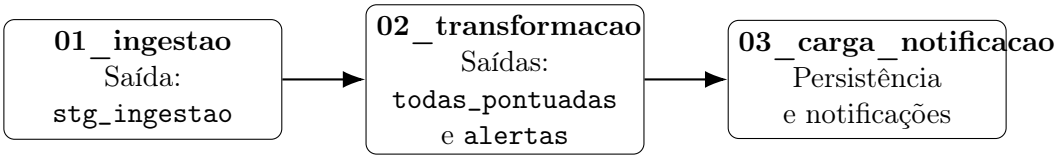



Figura 5.1: Sequência de execução do workflow orquestrador (`00_orquestrador`).

O encadeamento funciona da seguinte forma:

1. **Call Workflow Service :** chama `01_ingestao`, recolhendo o lote CSV e as mensagens da fila MQTT em PostgreSQL, aplicando a deduplicação e assegurando que a tabela de staging fica devidamente populada antes de prosseguir;

2. **Call Workflow Service :** chama `02_transformacao`, que executa a limpeza, as expressões regulares, a validação de Luhn, os cruzamentos relacionais e o cálculo de scores, expondo as saídas analíticas;

3. **Call Workflow Service :** chama `03_carga_notificacao.knwf`, recebendo as tabelas geradas e procedendo à escrita idempotente no data mart, exportação para ficheiros e envio do email de notificação.

A infraestrutura dos serviços de apoio é gerida localmente através de Docker Compose:

```
docker compose up -d
docker compose ps
docker compose down
```

Listing 5.1: Comandos para gestão dos serviços auxiliares com Docker Compose.

Além da execução na interface gráfica, a solução está apta para invocação em modo batch sem interface gráfica (*headless*), através da linha de comandos do KNIME:

```
knime -nosplash -application org.knime.product.KNIME_BATCH_APPLICATION \
     -workflowDir="dataintegration/workflows/00_orquestrador" -reset
```

Listing 5.2: Invocação do orquestrador via linha de comandos (Batch Mode).

Esta modalidade permite associar o job a ferramentas de agendamento do sistema operativo (como o utilitário `cron`), satisfazendo os requisitos de automatização industrial de processos de ETL.

# Capítulo 6

# Vídeo de demonstração

O vídeo de demonstração apresenta o pipeline completo em funcionamento prático, percorrendo os seguintes passos:

1. arranque dos contentores através do Docker Compose (`postgres`, `mosquitto`, `node-red`, `grafana` e `mailhog`);
2. verificação do simulador de transações no Node-RED e receção contínua de telemetria no PostgreSQL;
3. execução do job `00_orquestrador` no KNIME Analytics Platform, evidenciando o percurso pelas três etapas do processo;
4. validação dos resultados persistidos na base de dados (`stg_ingestao`, `mart_alertas` e `mart_resumo_diario`);
5. comprovação da notificação enviada no MailHog e dos ficheiros exportados em `data/output/`;
6. consulta dos painéis de monitorização no Grafana.

**Link do vídeo:** https://youtu.be/f$_d rmSF3Zd0$

Figura 6.1: QR code para acesso ao vídeo de demonstração.

# Capítulo 7

# Conclusão e trabalhos futuros

O trabalho prático permitiu conceber e concretizar uma solução completa de integração de dados orientada à deteção de fraude em transações com cartões bancários. A arquitetura implementada combinou com sucesso fontes heterogéneas (lote CSV, streaming MQTT, catálogos em JSON e regras em XML), garantindo a higienização dos registos com expressões regulares, o mascaramento de dados confidenciais e a validação algorítmica do número do cartão através da fórmula de Luhn.

A introdução de análises estatísticas individuais com base em $Z$-score associadas a um motor de regras de negócio ponderadas demonstrou elevada eficácia, permitindo triar 4 transações de alto risco entre mais de 10 400 movimentos processados, registando-as na camada de data mart, gerando ficheiros de exportação e emitindo notificações automáticas por email.

## 7.1 Decisões de âmbito e exclusões assumidas

Para assegurar a conclusão sólida das funcionalidades nucleares dentro do calendário letivo, foram tomadas opções intencionais de simplificação:

- **Taxas de Câmbio em Ficheiro JSON:** a obtenção de taxas cambiais recorre ao histórico estático em `cambio.json` em vez de uma API REST remota em tempo real, garantindo funcionamento autónomo e imune a falhas de rede ou limites de pedidos na demonstração;
- **Exportações Locais:** os artefactos finais são gerados no sistema de ficheiros local (em `data/output/`), prescindindo de ligações adicionais a servidores FTP/SFTP;
- **Formato de Exportação:** optou-se pela exportação nos formatos universais CSV e JSON em detrimento de formatos colunares como o Apache Parquet;
- **Agendamento Operacional:** validou-se a capacidade de execução autónoma em modo batch do KNIME via CLI, dispensando a instalação de orquestradores pesados como o Apache Airflow.

## 7.2 Trabalhos futuros

Como oportunidades de evolução futura do sistema, salientam-se:

1. integração de modelos de *Machine Learning* supervisionados e algoritmos de deteção não supervisionada de anomalias para complemento das regras manuais;
2. gravação sistemática de auditorias de execução na tabela `log_execucao` do PostgreSQL;

# Capítulo 8

# Referências bibliográficas

# Bibliografia


[1] Ferreira, L. e Ribeiro, Ó. *Integração de Sistemas de Informação: Trabalho Prático I — Enunciado*. Ano letivo 2026/27, Escola Superior de Tecnologia, Instituto Politécnico do Cávado e do Ave.

[2] KNIME AG. *KNIME Analytics Platform Documentation*. Disponível em: https://docs.knime.com/

[3] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponível em: https://www.postgresql.org/docs/

[4] International Organization for Standardization. *ISO/IEC 7812-1: Identification cards — Identification of issuers — Part 1: Numbering system*. Genebra, Suíça.

[5] Grafana Labs. *Grafana Documentation*. Disponível em: https://grafana.com/docs/grafana/latest/

[6] Docker Inc. *Docker Compose Documentation*. Disponível em: https://docs.docker.com/compose/

[7] OpenJS Foundation. *Node-RED Documentation*. Disponível em: https://nodered.org/docs/

[8] Eclipse Foundation. *Eclipse Mosquitto Documentation*. Disponível em: https://mosquitto.org/documentation/

[9] MailHog. *MailHog: Web and API based SMTP testing*. Disponível em: https://github.com/mailhog/MailHog